

IPW

Introdução à Programação Web

Resumo Analítico Completo · Todas as Lessons

LEIC · ISEL

Índice

Lesson 00 — Apresentação da Unidade Curricular

- Descrição, bibliografia, avaliação

Lesson 01 — Introdução ao JavaScript — Valores, Tipos e Operadores

- Tipos primitivos, operadores, coerção

Lesson 02 — Estrutura de Programas

- Expressões, variáveis, controlo de fluxo, funções

Lesson 03 — Funções

- Declaração, escopo, closures, recursão

Lesson 04 — Estruturas de Dados: Objetos e Arrays

- Objetos, arrays, JSON, métodos

Lesson 05 — Higher-Order Functions

- forEach, filter, map, reduce

Lesson 06 — Exercícios Propostos (JS)

- Histograma, Student, prototypes

Lesson 07 — Módulos, Node.js, NPM e Testes

- ESM vs CJS, import/export, Mocha

Lesson 08 — Programação Assíncrona

- Callbacks, Promises, async/await

Lesson 09 — Uso de Promises

- fs/promises, fetch API, Promise.all/any/race

Lesson 10 — Introdução à Web

- WWW, URI/URL, HTTP, métodos e status codes

Lesson 11 — Framework Express

- Routing, CRUD, middleware básico

Lesson 12 — Web API e OpenAPI

- REST, OpenAPI spec, Tasks API

Lesson 13 — Melhorias da Web API (Parte 1)

- Organização, compliance OpenAPI, token 401

Lesson 14 — Melhorias da Web API (Parte 2)

- Módulo de dados assíncrono, gestão de erros

Lesson 15 — Express Middleware

- Built-in, third-party, CORS, error handling

Lesson 16 — Elasticsearch

- Índices, documentos, CRUD via HTTP

Lesson 17 — Dependency Injection

- export default, init functions, diagrama Tasks API

Lesson 18 — HTML e CSS (em Português)

- Estrutura HTML, formulários, seletores CSS, Bootstrap

Lesson 19 — Template Engine com Express

- Handlebars/HBS, partials, helpers, views

Lesson 20 — Autenticação com Passport

- Sessions, cookies, login/logout, signup

Lesson 00

Apresentação da Unidade Curricular

Contexto Geral

A UC **IPW – Introdução à Programação Web** está inserida no 3.º semestre do LEIC/ISEL. O plano curricular está disponível em https://www.isel.pt/sites/default/files/FUC_LEIC/3sem/IPW_LEIC.pdf.

Avaliação

Fórmula final: **40% [T] + 60% [E+P]**

Componentes:

- **[T] Teste** — realizado no período de exame normal; pode ser repetido na época de recurso.
- **[E] Série de Exercícios** — entrega individual.
- **[P] Trabalho Prático** — 3 partes; nota individual determinada em discussão oral.

Condições de Aprovação

- Média ponderada de todas as componentes > 9,5.
- Cada parte do trabalho entregue dentro do prazo e com nota > 9,5.
- Nota do teste ≥ 9,5.

Época Especial

Componente teórica [T]: teste escrito na data da época especial. Componente prática [P]: realizada individualmente, com enunciado autónomo, entrega obrigatória antes do teste.

Bibliografia Principal

- **JavaScript:** Eloquent JavaScript (4.^a ed.) — Marijn Haverbeke; MDN Web Docs; Secrets of the JavaScript Ninja (2.^a ed.).
- **Node.js:** Node.js 8 the Right Way; Nodeschool; The Art of Node.
- **Web/HTTP:** Architecture of the WWW (W3C); RFC 2616.
- **HTML/CSS:** Learn to Code HTML & CSS (Shay Howe).
- **Geral:** devdocs.io; Open API Specification.

Lesson 01 Introdução ao JavaScript — Valores, Tipos e Operadores

Contexto e Origem

- Criado em **1995** por **Brendan Eich** para o Netscape Navigator.
- Adotado por todos os grandes browsers. Não tem relação com Java (nome é marketing).
- Padronizado como **ECMAScript** para uniformidade entre browsers.
- Versões relevantes: ES3 (2000–2010), ES5 (2009), **ES6/2015** (grande atualização), atualizações anuais desde então.
- Fora dos browsers: MongoDB, CouchDB, **Node.js** (servidor/desktop).

Tipos Primitivos (7 tipos)

Tipo	Descrição	Exemplo
Number	Ponto flutuante (único tipo numérico)	25, 3.14, NaN
String	Sequência de caracteres	"hello", `template`
Boolean	Verdadeiro ou falso	true, false
BigInt	Inteiro arbitrariamente grande	10000n
Symbol (ES6)	Valor único e imutável	Symbol('id')
Undefined	Variável declarada sem valor	let x; // undefined
Null	Ausência intencional de valor	let y = null

Operadores

Aritméticos

```
+, -, *, /, % (resto), ** (potência)
```

Comparação

```
> &lt; >= &lt;= == != (com coerção) === !== (sem coerção – preferir sempre)
```

Lógicos

```
&& (and) || (or) ! (not)
```

Ternário

```
A ? B : C // resultado é B se A for true, C caso contrário
```

Coerção de Tipos (Type Coercion)

JS é **dinamicamente tipado** e **fracamente tipado**: converte automaticamente tipos para 'encaixar' operações.

```
8 * null // → 0
"5" - 1 // → 4
"5" + 1 // → "51" (concatenação!)
"five" * 2 // → NaN
false == 0 // → true (coerção)
false === 0 // → false (sem coerção – PREFERIR)
null != undefined // → false
null !== undefined // → true
```

■ Usar sempre `===` e `!==` para evitar bugs por coerção implícita.

typeof

```
typeof 42 // → "number"
typeof "hello" // → "string"
typeof true // → "boolean"
typeof undefined // → "undefined"
typeof null // → "object" ← quirk histórico do JS!
```

[VERIFICAR] `typeof null` retornar `'object'` é um bug histórico mantido por retrocompatibilidade.

Lesson 02

Estrutura de Programas

Expressões vs. Statements

- **Expressão:** fragmento de código que produz um valor. Ex: $2+3$, $5*(2+8)$.
- **Statement:** realiza uma ação (atribuição, execução de comando). Terminado por ';' (opcional em alguns contextos).

Variáveis

Keyword	Escopo	Reatribuível	Re-declarável
let	Bloco	Sim	Não (no mesmo escopo)
const	Bloco	Não	Não
var	Função/Global	Sim	Sim (evitar)

Regras de Nomenclatura

- Sequência de letras, dígitos (não no início), \$ ou _.
- Convenção: **camelCase** (ex: variableNameExample).
- JavaScript é **case-sensitive**.

Controlo de Fluxo

Condicional

```
if (condition) {  
  // bloco  
} else {  
  // bloco  
}
```

Loops

```
// while  
while (condition) { /* ... */ }  
  
// do-while  
do { /* ... */ } while (condition);  
  
// for  
for (let i = 0; i < n; i++) { /* ... */ }
```

- **break:** sai imediatamente do loop.
- **continue:** salta para a próxima iteração.

Switch

```
switch (expression) {  
  case val1:  
    // ...  
    break; // ← obrigatório para não cair no próximo case  
  default:  
    // ...  
}
```

Funções (visão geral)

- Em JS, uma função é um **valor** (objeto de primeira classe).
- Se não houver return, a função devolve undefined.

```
console.log(Math.max(2, 4)); // → 4  
console.log((Math.max(2,4)*3) >= 10); // → true
```

Comentários

```
// Comentário de uma linha  
/* Comentário  
de várias linhas */
```


Lesson 03

Funções

Três Formas de Definir Funções

1. Declaration Notation

```
function square(n) {  
  return n * n;  
}  
console.log(square(7)); // → 49  
// Nota: pode ser chamada ANTES da declaração (hoisting)
```

2. Function Expression (como valor)

```
const square = function (x) {  
  return x * x;  
};
```

3. Arrow Function (ES6)

```
const square = (x) => { return x * x; };  
// Forma compacta (expressão única, 1 argumento):  
const square = x => x * x;
```

Escopos

Tipo	Keyword	Contexto
Global	var, let, const	Fora de qualquer função/bloco
Função	var	Dentro de uma função
Bloco	let, const	Dentro de {} (if, for, etc.)

```
let x = 10; // global  
if (x > 0) {  
  let y = 20; // local ao bloco  
  var z = 30; // global (var ignora bloco!)  
}  
// y não existe aqui; z existe
```

Call Stack

Cada chamada a uma função guarda o contexto atual no **call stack**. Quando a função termina, o contexto é removido. Se a stack exceder a memória → **RangeError: Maximum call stack size exceeded** (stack overflow).

Recursão

```
function power(base, exponent) {  
  if (exponent == 0) return 1;  
  return base * power(base, exponent - 1);  
}  
console.log(power(2, 3)); // → 8
```

Closure

Uma **closure** é uma função que referencia variáveis do âmbito local que a envolve, mantendo-as 'vivas' após o retorno da função exterior.

```
function setCounter() {  
  let c = 0;  
  return function () {  
    c += 1; return c;  
  };  
}  
const counter = setCounter();  
counter(); // → 1  
counter(); // → 2  
counter(); // → 3
```

A variável 'c' persiste entre chamadas graças à closure.

Lesson 04

Estruturas de Dados: Objetos e Arrays

Objetos

- **Definição:** coleção de propriedades (nome: valor). Valor pode ser qualquer tipo, incluindo funções.
- **Acesso:** `obj.prop` ou `obj['prop']`.
- Propriedades nomes são strings. Iteração com **for-in**.

```
const point1 = {x: 3.2, y: 5.9};
point1.color = 'green'; // adicionar propriedade
delete point1.color; // remover propriedade
'x' in point1; // → true
```

this

Dentro de um método de objeto, **this** refere-se ao próprio objeto. Em funções standalone, **this** aponta para o global ou undefined (strict mode).

Constructor Function e Prototype

```
function Point2D(x, y) {
  this.x = x; this.y = y;
}
let p = new Point2D(3.2, 4.9);

// Adicionar via prototype (afeta TODOS os objetos já criados e futuros):
Point2D.prototype.color = undefined;
Point2D.prototype.setColor = function(c) { this.color = c; };
```

Mutabilidade

- **Primitivos:** imutáveis (não se pode mudar o valor interno, só reatribuir a variável).
- **Objetos:** mutáveis (propriedades podem ser alteradas mesmo com const).

const com objetos impede reatribuição da variável, mas não mutação do objeto.

Arrays

```
const arr = [1, 2, 3];
arr[0]; // → 1 (índice começa em 0)
arr[arr.length-1]; // → último elemento
```

Métodos Principais

Método	Descrição
push(v)	Adiciona ao fim
pop()	Remove e retorna o último
unshift(v)	Adiciona ao início
shift()	Remove e retorna o primeiro
indexOf(v)	Primeiro índice de v (ou -1)
lastIndexOf(v)	Último índice de v
slice(start, end)	Subarray [start, end[
forEach(fn)	Aplica fn a cada elemento

filter(pred)	Filtra por predicado
map(fn)	Transforma cada elemento
reduce(fn)	Combina todos os elementos

Métodos Úteis de String

- trim(), padStart(n, c), repeat(n), split(sep), join(sep), substring(s,e), concat()

JSON

```
JSON.stringify(obj) // objeto → string  
JSON.parse(str) // string → objeto
```

Outros Conceitos

- **Rest parameters:** function f(...args) — recebe número indefinido de argumentos como array.
- **Optional chaining:** obj?.prop — retorna undefined se prop não existir.
- **Destructuring:** const {x, y} = obj; / let [a,b,...rest] = arr;
- **Object.assign, Object.entries, Object.keys, Object.values, Object.fromEntries.**
- **call() / apply():** invocam função com this explícito.

Lesson 05

Higher-Order Functions

Definição

Higher-order functions: funções que recebem outras funções como argumento ou as retornam.

forEach

Aplica uma callback a cada elemento. Não retorna valor.

```
['Ana', 'Bob'].forEach(elem => console.log(elem));
```

filter

Filtra elementos segundo um predicado. Retorna novo array.

```
[1,4,3,20,11].filter(n => n % 2 == 0); // → [4, 20]  
[1,4,3,20,11].filter(n => n > 10); // → [20, 11]
```

map

Aplica uma função a cada elemento. Retorna novo array.

```
[1,4,3].map(n => n*n); // → [1, 16, 9]
```

reduce

Combina todos os elementos com uma função acumuladora. Retorna um valor.

```
[1,2,4].reduce((acc, n) => acc + n); // → 7
```

Composição de Operações

```
// filter + map  
[1,2,3,4].filter(n => n%2==0).map(n => n*n); // → [4, 16]  
  
// map + reduce (soma dos quadrados)  
[1,2,3,4].map(n => n*n).reduce((a,b) => a+b); // → 30
```

Ligação com Lesson Anterior

As higher-order functions são possíveis porque em JS as funções são **valores de primeira classe** (Lesson 03 e 04) — podem ser passadas como argumentos e retornadas.

Lesson 06

Exercícios Propostos (JavaScript)

Exercício 1 — Histograma com Closure

Implementar função que recebe string e retorna histograma de caracteres (não case-sensitive) como objeto literal. Deve usar **closure** para manter histórico de chamadas anteriores.

```
histogram('Era uma vez');  
// → { E: 2, R: 1, A: 2, ' ': 2, U: 1, M: 1, V: 1, Z: 1 }  
histogram('uma casa no campo');  
// → { E:2, R:1, A:6, ' ':5, U:2, M:3, V:1, Z:1, C:2, S:1, N:1, O:2, P:1 }  
// (acumula com chamadas anteriores!)
```

Exercício 2 — Constructor Function Student

- Criar constructor function **Student** com: name, lista de surnames, lista de scores (0–20).
- Método para modificar a lista de scores.
- 2a. Criar 4 objetos Student e adicionar a array.
- 2b. Adicionar ao **prototype** função que calcula média **sem usar loops ou forEach**.
- 2c. Função **highPerformance**: dado array de Students, retornar array com {name, average} dos que têm média ≥ 17 .
- 2d. Reimplementar 2c sem loops ou forEach.

Exercício 3 — Soma de Arrays via Prototype

```
let arr1 = [1, 4, 3, 7];  
let arr2 = [5, 0, 2, 1];  
arr1.sumArray(arr2); // → [6, 4, 5, 8]  
// Adicionar sumArray ao Array.prototype
```

Exercício 4 — Override de console.info

Sobrescrever console.info para enumerar cada print (adicionar número sequencial ao início da string).

Lesson 07

Módulos, Node.js, NPM e Testes

Módulos JavaScript

Tipo	Extensão	Import	Export
ESM (ES Module)	.mjs	import {x} from './file.mjs'	export function x(){} export {x}
CJS (CommonJS)	.cjs	const x = require('./file')	module.exports = x

Nesta UC usa-se exclusivamente ESM.

Import / Export (ESM)

```
// dayname.mjs
export function dayName(n) { return names[n]; }
export function dayNumber(name) { return names.indexOf(name); }

// uso:
import {dayName} from './dayname.mjs';
import {dayName as nomeDoDia} from './dayname.mjs'; // alias
import * as dn from './dayname.mjs'; // importar tudo
```

NPM — Comandos Básicos

```
npm init # cria package.json
npm install express # instala dependência
npm install mocha --save-dev # dependência de desenvolvimento
npm test # executa testes
```

Testes com Mocha

- Ficheiros de teste na pasta **test/** por defeito.
- **describe()**: agrupa testes com uma descrição.
- **it()**: teste individual. Estrutura: **Arrange** → **Act** → **Assert**.
- **assert**: módulo `node:assert`.

```
describe('Simple test suite', () => {
  it('Test 1', () => {
    // Arrange
    // Act
    // Assert (ex: assert.strictEqual(result, expected))
  });
});
```

Exercício — Teste do Histograma

```
'aaaa' → { a: 4 }
'bbbb' → { a: 4, b: 4 } (closure acumula!)
'BBb' → { a: 4, b: 7 }
```

Lesson 08

Programação Assíncrona

Síncrono vs. Assíncrono

Modelo	Comportamento
Síncrono	Função longa bloqueia o programa até terminar
Assíncrono	Inicia ação; programa continua; callback/promise notifica quando termina

JavaScript é Single-threaded

Não há paralelismo real. Funções assíncronas (I/O ficheiro, rede) interrompem o processamento; enquanto esperam, o CPU pode executar outras operações.

Abordagem 1: Callback

```
setTimeout(() => console.log('Ok!'), 1000);  
// setTimeout retorna imediatamente e regista o callback para +1s
```

Problema: callback hell — difícil de ler, depurar e tratar erros.

Abordagem 2: Promise

Promise: objeto que representa o resultado futuro (ou falha) de uma operação assíncrona.

Estados da Promise

- **pending:** estado inicial.
- **fulfilled:** operação completada com sucesso.
- **rejected:** operação falhou.

Criar e Consumir

```
const p = new Promise((resolve, reject) => {  
  if (ok) resolve(value);  
  else reject(error);  
});  
  
p.then(onFulfilled) // sucesso  
  .then(onFulfilled2)  
  .catch(onRejected); // falha
```

Async / Await (ES2017)

```
async function f1() {  
  try {  
    const x = await somePromise(5);  
    console.log('x =', x);  
  } catch(e) {  
    console.error(e);  
  }  
}  
  
// async faz a função retornar sempre uma Promise  
// await suspende a função até a Promise resolver (sem bloquear o event loop)
```

Equivalência then ↔ await


```
// Com then:
resolveAfter3Sec(10).then(x => console.log(x));

// Com await (dentro de async):
const x = await resolveAfter3Sec(10);
console.log(x);
```

Lesson 09

Uso de Promises — fs e fetch

1. Módulo fs/promises (Node.js)

- **readFile(path)** → Promise<Buffer>
- **writeFile(path, data)** → Promise<undefined>

```
import { readFile, writeFile } from 'node:fs/promises';
readFile('input.txt')
  .then(buf => buf.toString().split('\n')[0])
  .then(line => writeFile('output.txt', line))
  .catch(err => console.error(err));
```

2. Fetch API

- Global em Node.js ≥ 18 e browsers.
- Sintaxe: **fetch(url[, options])** → Promise<Response>

Objeto Response

- status, headers, body
- **text()** → Promise<String>
- **json()** → Promise<Object>

```
fetch(URL)
  .then(resp => resp.json())
  .then(data => console.log(data))
  .catch(err => console.log('Error'));
```

Concorrência de Promises

Método	Fulfills quando	Rejeita quando
Promise.all(iter)	TODAS fulfilladas	QUALQUER rejeita
Promise.any(iter)	QUALQUER fulfillada	TODAS rejeitam → AggregateError
Promise.race(iter)	QUALQUER fulfillada	QUALQUER rejeita (o que acontecer primeiro)

Exercícios

- Fetch de URL do Eloquent JS e extrair título entre e .
- Fetch e contar ocorrências de uma palavra.
- Versão async/await do fs-promise-chaining.
- Fetch de múltiplos URLs (versões com Promise explícita e async/await).
- Ler JSON de ficheiro fruits.json e escrever links das imagens num novo ficheiro.

Lesson 10

Introdução à Web

WWW — Conceitos Fundamentais

"The WWW is an information space in which items of interest, referred to as resources, are identified by global identifiers called URIs." — W3C

- Criada por **Tim Berners-Lee** em 1990 (proposta em 1989 no CERN).
- 1991: publicação do HTTP, HTML e primeiro browser (WorldWideWeb).
- 1994: fundação do **W3C**. 1995: JavaScript.

Arquitetura Web

- **Identificação**: recursos identificados por URIs.
- **Interação**: protocolo HTTP.
- **Representação**: formato HTML, JSON, etc.

URI — Uniform Resource Identifier

```
URI = scheme ':' [ '://' authority ] path [ '?' query ] [ '#' fragment ]  
authority = [userinfo '@'] host [ ':' port ]
```

Exemplo:

```
foo://example.com:8042/over/there?name=ferret#nose  
 \  / \_____/ \_____/ \_____/ \_____/ \_____  
scheme authority path query fragment
```

- **URL**: URI que localiza recursos na Internet (host resolve por DNS).
- **URN**: identificador persistente independente de localização. Ex: urn:isbn:...

HTTP — Hypertext Transfer Protocol

- Modelo **cliente-servidor**, protocolo **stateless**, padrão **request-response**.
- Porta padrão: 80 (HTTP), 443 (HTTPS).

Métodos HTTP

Método	Descrição
GET	Obter representação de recurso
POST	Submeter entidade (cria/altera estado)
PUT	Substituir recurso completo
DELETE	Eliminar recurso
HEAD	Como GET mas sem body
OPTIONS	Opções de comunicação permitidas
PATCH	Modificação parcial

Status Codes HTTP

Intervalo	Classe	Exemplos
1xx	Informativo	100 Continue
2xx	Sucesso	200 OK, 201 Created

3xx	Redireção	302 Found, 301 Moved Permanently
4xx	Erro do cliente	400 Bad Request, 401 Unauthorized, 404 Not Found
5xx	Erro do servidor	500 Internal Server Error

Versões HTTP

- HTTP/1.0 [RFC 1945]: uma request por conexão TCP.
- HTTP/1.1 [RFC 2616]: persistente com pipeline (múltiplas requests na mesma conexão).
- HTTP/2 [RFC 9113]: multiplexing, framing binário, priorização.
- HTTP/3 [RFC 9114]: protocolo QUIC.

Lesson 11 Framework Express

Express — Visão Geral

- Framework de routing e middleware para Node.js com funcionalidade mínima.
- Instalação: `npm install express`

```
import express from 'express';
const app = express();
app.listen(8000, () => console.log('Listening on port 8000'));
```

Routing

```
app.METHOD(PATH, HANDLER)

app.get('/tasks', getAllTasks);
app.post('/tasks', addTask);
app.put('/tasks/:id', updateTask);
app.delete('/tasks/:id', deleteTask);
app.get('/user/:id', (req, res) => res.send(req.params.id));
```

Request e Response

Objeto	Propriedade/Método	Descrição
req	req.params.id	Parâmetro de rota (:id)
req	req.body	Corpo da request (JSON)
req	req.query	Query string
res	res.send(data)	Envia resposta
res	res.json(obj)	Envia JSON
res	res.status(code)	Define status HTTP
res	res.end()	Termina resposta sem dados

Middleware — `express.json()`

```
app.use(express.json()); // parseia body JSON nas requests POST/PUT
```

Tasks API — CRUD em Memória

Operação	HTTP Method	URI
getAllTasks	GET	/tasks
getTask	GET	/tasks/:id
addTask	POST	/tasks
updateTask	PUT	/tasks/:id
deleteTask	DELETE	/tasks/:id

Nodemon

```
npm install -g nodemon
npm run start-dev # reinicia automaticamente ao guardar ficheiros
```

Lesson 12

Web API e OpenAPI

REST — Arquitetura

REST (Representational State Transfer) é um estilo arquitetural para sistemas hipermedia distribuídos, definido na dissertação de Roy Fielding (2000).

6 Restrições REST

- **Client-Server:** separação de responsabilidades.
- **Stateless:** cada request contém toda a informação necessária.
- **Cache:** respostas devem indicar se são cacheáveis.
- **Uniform Interface:** interface consistente e uniforme.
- **Layered System:** cada componente não vê além da camada imediata.
- **Code-On-Demand** (opcional): cliente pode executar código enviado pelo servidor.

REST ≠ HTTP. REST é estilo arquitetural; HTTP é protocolo de comunicação.

OpenAPI Specification

- Formato YAML ou JSON para descrever REST APIs.
- Ferramentas: Swagger UI (documentação interativa), geração de código.

Estrutura Principal

```
openapi: 3.0.1
info:
  title: Tasks API
  version: 1.0.0
servers:
  - url: http://localhost:8000
security:
  - bearerAuth: []
paths:
  /tasks:
    get: ...
    post: ...
  /tasks/{taskId}:
    get: ...
    put: ...
    delete: ...
components:
  schemas:
    NewTask: ...
    Task: ...
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
```

Autenticação por Token (Bearer)

```
Authorization: Bearer f1d1cdb9c-97f0-41c4-b206-051250684b19
```

- Token é UUID gerado por `crypto.randomUUID()` (módulo `node:crypto`).

Integração Swagger UI com Express

```
npm install swagger-ui-express ymljs
```

```
import swaggerUi from 'swagger-ui-express';  
import yaml from 'yamljs';  
const doc = yaml.load('./docs/tasks-api.yaml');  
app.use('/doc', swaggerUi.serve, swaggerUi.setup(doc));  
// Acesso: http://localhost:8000/doc
```

Lesson 13

Melhorias da Web API — Parte 1

1. Organização em Diretórios

```
example-tasks/  
  ■■■ tasks-server.mjs  
  ■■■ docs/tasks-api.yaml  
  ■■■ commons/  
  ■■■ data/tasks-data.mjs  
  ■■■ services/  
    ■ ■■■ tasks-services.mjs  
    ■ ■■■ users-services.mjs  
  ■■■ web/api/  
    ■■■ tasks-web-api.mjs  
    ■■■ users-web-api.mjs
```

2. Compliance com OpenAPI

Cada função do web-api deve retornar respostas conformes com o OpenAPI spec. Exemplos:

- addUser: retornar status 201 + { token: uuid }
- addTask: retornar status 201 + { status: 'Task X was added!', task: {...} }
- getTask: retornar task ou status 404 + { code, description }

3. Status 401 — Token Ausente

Implementação de função **processRequest** que envolve cada handler e verifica token:

```
function processRequest(operation) {  
  return function(req, res) {  
    const token = getToken(req);  
    if (!token) {  
      res.status(401);  
      return res.send({ code: 3, error: 'Missing Token' });  
    }  
    req.userToken = token;  
    return operation(req, res);  
  };  
}  
  
export const getAllTasks = processRequest(local_getAllTasks);  
export const getTask = processRequest(local_getTask);  
// ... etc.
```


Lesson 14

Melhorias da Web API — Parte 2

4. Módulo de Dados Assíncrono

Todas as funções do módulo de dados retornam **Promises** (em vez de valores síncronos).

```
export function getTask(taskId) {
  return Promise.resolve(TASKS.find(t => t.id == taskId));
}
export function deleteTask(idTask) {
  return new Promise((resolve, reject) => {
    const idx = TASKS.findIndex(t => t.id == idTask);
    if (idx != -1) { resolve(TASKS.splice(idx,1)[0]); }
    else reject(errors.TASK_NOT_FOUND(idTask));
  });
}
```

5. Gestão de Erros

Módulo commons/errors.mjs — Códigos Internos

Código	Nome	HTTP Status
1	MISSING_PARAMETER	400 Bad Request
2	INVALID_PARAMETER	400 Bad Request
3	INVALID_BODY	400 Bad Request
4	TASK_NOT_FOUND	404 Not Found
5	USER_NOT_FOUND	401 Unauthorized
6	NOT_AUTHORIZED	401 Unauthorized
7	MISSING_TOKEN	401 Unauthorized
default	—	500 Internal Server Error

Fluxo de Erros

- Funções de serviço retornam **Promise.reject(errors.ERRO())** em caso de falha.
- Módulo **errors-to-http-responses.mjs** converte erro interno em { status, body }.
- Em processRequest, o **.catch()** chama o converter e envia resposta adequada.

Route com ? para parâmetro opcional

```
app.delete('/tasks/:taskId?', tasksAPI.deleteTask);
app.put('/tasks/:taskId?', tasksAPI.updateTask);
// Permite tratar ausência de taskId com erro 400 em vez de 404 genérico
```

Safe e Idempotent HTTP Methods

Método	Safe?	Idempotent?
GET	Sim	Sim
HEAD	Sim	Sim
PUT	Não	Sim
DELETE	Não	Sim

POST	Não	Não
PATCH	Não	Não (geralmente)

Safe = só leitura; Idempotent = N chamadas idênticas = resultado do estado do servidor igual a 1 chamada.

Lesson 15

Express Middleware

Definição

"An Express application is essentially a series of middleware function calls."

Middleware functions têm acesso a **req**, **res** e **next**. Se não terminam o ciclo (com `send/json`), **devem chamar `next()`**.

Tipos de Middleware

Tipo	Descrição
Built-in	<code>express.static</code> , <code>express.json</code> , <code>express.urlencoded</code>
Third-party	<code>cors</code> , <code>morgan</code> , etc. (instalados via npm)
Application-level	<code>app.use()</code> ou <code>app.METHOD()</code>
Error-handling	4 parâmetros: (<code>err</code> , <code>req</code> , <code>res</code> , <code>next</code>)
Router-level	Associado a instância de <code>express.Router()</code>

Ordem Importa!

Middleware é executado sequencialmente na ordem em que é registrado. O error-handling middleware deve ficar no final.

```
app.use(express.json()); // 1.º – parseia body
app.use(cors()); // 2.º – CORS
app.get('/tasks', handler); // 3.º – rotas
app.use(errorHandler); // último – error handling
```

CORS — Cross-Origin Resource Sharing

Por razões de segurança, browsers bloqueiam requests cross-origin iniciadas por scripts. CORS usa headers HTTP (ex: `Access-Control-Allow-Origin`) para permitir exceções.

```
npm install cors

import cors from 'cors';
app.use(cors()); // permite todos os origins
```

Error-Handling Middleware

```
// Deve ter exatamente 4 parâmetros:
function errorHandler(err, req, res, next) {
  console.error(err.stack);
  res.status(500).send('Something broke!');
}
app.use(errorHandler);

// Para async functions, passar erro ao next():
readFile(req, res, next) {
  return fs.readFile('/file')
    .then(data => res.send(data))
    .catch(err => next(err));
}
```

Lesson 16 Elasticsearch

Definição

- Motor de busca e análise open-source, distribuído, construído sobre **Apache Lucene**.
- Características: full-text search, analytics em tempo real, escalabilidade, API RESTful.

Arquitetura

Conceito	Equivalente SQL	Descrição
Node	—	Instância Elasticsearch
Cluster	—	Conjunto de nodes
Index	Database	Namespace lógico de documentos similares
Document	Row	JSON com campos (fields)
Shard	—	Divisão do índice para distribuição
Replica	—	Cópia de shard para alta disponibilidade

URL Base e Configuração

```
# Por defeito:
http://localhost:9200

# Desativar segurança para testes (config/elasticsearch.yml):
xpack.security.enabled: false
xpack.security.enrollment.enabled: false
```

Operações CRUD — HTTP

```
# Criar documento (ID auto-gerado):
POST /books/_doc
Content-Type: application/json
{ "name": "The Handmaids Tale", "author": "Margaret Atwood", ... }

# Listar todos:
GET /books/_search

# Filtrar:
POST /books/_search { "query": { "match": { "author": "Margaret Atwood" } } }

# Obter por ID:
GET /books/_doc/nnq00ZMBYf62TULrV1M3

# Atualizar por ID:
PUT /books/_doc/nnq00ZMBYf62TULrV1M3 { ... }

# Eliminar por ID:
DELETE /books/_doc/oHro0ZMBYf62TULrT10G
```

Integração com Tasks API

- Separar em data/elastic/ e data/memory/ para facilitar troca do backend.
- Versão v1.2.0 integra Elasticsearch como base de dados persistente.

Lesson 17

Dependency Injection

Definição

Dependency Injection: padrão de design em que um módulo recebe as suas dependências de uma fonte externa em vez de as criar internamente.

export default + init Function

Cada módulo exporta uma única função init (como default export) que recebe as suas dependências como parâmetros e retorna um objeto com as funções públicas.

```
// module2.mjs
export default function init() {
  return { printHello };
  function printHello(name) { console.log('Hello ' + name); }
}

// module1.mjs
export default function init(m2, m3) {
  if (!m2) throw 'Error: missing m2!';
  return { printAll };
  function printAll() { m2.printHello('Ana'); }
}

// main.mjs – dependências explícitas e centralizadas:
import initM1 from './module1.mjs';
import initM2 from './module2.mjs';
const m2 = initM2();
const m1 = initM1(m2);
m1.printAll();
```

Vantagens

- Todas as dependências da aplicação são visíveis num único módulo (main).
- Facilita testes unitários (substituição de dependências por mocks).
- Módulos mais desacoplados e reutilizáveis.

Tasks API — Diagrama de Dependências

```
tasks-server.mjs
■ ■ → tasks-web-api.mjs (init(tasksServices))
■ ■ ■ → tasks-services.mjs (init(tasksData, usersServices))
■ ■ ■ → tasks-data-elastic.mjs (init())
■ ■ ■ → users-services.mjs (init(usersData))
■ ■ ■ → users-data-elastic.mjs (init())
■ ■ ■ → users-web-api.mjs (init(usersServices))
```

Versões do Código

- v1.0.0: compliance OpenAPI + token check.
- v1.1.0: módulo de dados assíncrono + gestão de erros.
- v1.2.0: integração Elasticsearch.
- v1.3.0: dependency injection.
- v2.0.0: Tasks Web Site (template engine HBS).

Lesson 18 HTML e CSS — Visão Geral

Esta lesson está nos apontamentos em português.

HTML — HyperText Markup Language

- **Marcação:** anotação de conteúdo para alterar propriedades (fonte, cor, tamanho...).
- **Hipertexto:** texto com referências a outros textos, imagens, sons.

Estrutura de um Documento

```
<!DOCTYPE html>
<html>
<head> <!-- metadados, scripts, estilos -->
<title>Título</title>
</head>
<body> <!-- conteúdo visível -->
<h1>Título</h1>
<p>Parágrafo</p>
</body>
</html>
```

Tags Principais

Tag	Propósito
h1–h6	Cabeçalhos de secções
p	Parágrafo
a href='...'	Hiperlink
table, tr, th, td	Tabelas
ul/ol, li	Listas (não ordenada/ordenada)
div	Divisão/contêiner genérico
span	Trecho inline
img src='...'	Imagem
form	Formulário
input type='...'	Campo de entrada
header, footer, main, nav, section, article	Elementos semânticos

Formulários

```
<form action='/endpoint' method='POST'>
<label for='nome'>Nome:</label>
<input type='text' id='nome' name='nome'>
<input type='submit' value='Enviar'>
</form>

# Métodos suportados pelo HTML form: GET e POST
# POST: dados no body (application/x-www-form-urlencoded)
# GET: dados na URL (query string)
```

CSS — Cascading Style Sheets

```
/* Sintaxe geral */
seletor { propriedade: valor; }

/* Exemplos de seletores */
p { color: blue; } /* todos os <p> */
#meuId { color: red; } /* elemento por id */
.minhaClasse { color: green; } /* por classe */
div > p { font-size: 12px; } /* filhos diretos */
p:first-child { font-weight: bold; }
```

Bootstrap 5

- Framework front-end para design responsivo e mobile-first.
- Usar classes CSS predefinidas nos atributos class das tags HTML.

```
<link href='https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css'
rel='stylesheet'>
<script
src='https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js'></script>
```

Lesson 19 Template Engine com Express — Handlebars

Template Engine

Permite usar ficheiros de template HTML com variáveis. O engine substitui as variáveis por valores reais e transforma o template em HTML final.

- Exemplos: **Handlebars (HBS)**, EJS, Pug.

Ficheiros Estáticos

```
app.use(express.static('public')); // serve CSS, imagens, JS client-side
```

Handlebars — Sintaxe

Sintaxe	Função
{{var}}	Interpolação com HTML-escaping
{{{var}}}	Interpolação sem escape
{{> partial}}	Incluir partial
{{#if cond}}...{{else}}...{{/if}}	Condicional
{{#each list}}...{{/each}}	Iteração (this = elemento, @index = índice)
{{#with obj}}...{{/with}}	Mudar contexto de avaliação

Configuração HBS no Express

```
import hbs from 'hbs';
import path from 'path';
import url from 'url';

const CURRENT_DIR = url.fileURLToPath(new URL('.', import.meta.url));
const PATH_VIEWS = path.join(CURRENT_DIR, 'views');
const PATH_PARTIALS = path.join(PATH_VIEWS, 'partials');

app.set('views', PATH_VIEWS);
app.set('view engine', 'hbs');
hbs.registerPartials(PATH_PARTIALS);
hbs.registerHelper('upperCase', str => str.toUpperCase());
```

Render de Views

```
app.get('/', (req, res) => {
  res.render('home', { user: {name:'Ana', surname:'Silva'} });
});
// Procura views/home.hbs e injeta o objeto como contexto
```

Layout Global (layout.hbs)

```
<!-- views/layout.hbs -->
<!DOCTYPE html><html><head>...</head>
<body>{{{body}}}</body></html>
<!-- {{{body}}} é substituído pelo conteúdo da view específica -->
```

Tasks Web Site — Rotas


```
app.get('/site/tasks', tasksSite.getAllTasks);
app.get('/site/tasks/:taskId', tasksSite.getTask);
app.post('/site/tasks', tasksSite.addTask);
app.post('/site/tasks/:taskId/delete', tasksSite.deleteTask);
app.post('/site/tasks/:taskId/update', tasksSite.updateTask);
// DELETE e PUT usam POST por limitação dos HTML forms
```

Para DELETE/PUT em forms HTML, usa-se POST. Futuramente, fetch no client-side permitirá usar os métodos corretos.

Lesson 20

Autenticação com Passport

Passport

Passport é middleware de autenticação para Node.js. Autentica utilizadores e pode associar-se a uma **sessão web** Express.

```
npm install passport express-session
```

Sessões e Cookies

- Uma sessão é estabelecida com **HTTP cookies** no browser.
- O browser envia o cookie em cada request subsequente.
- O cookie contém tipicamente um ID de sessão aleatório.

```
import session from 'express-session';

const sessionHandler = session({
  secret: 'a-secret',
  resave: false,
  saveUninitialized: false,
});
app.use(sessionHandler);
app.use(passport.session());
```

Serialize / Deserialize

```
passport.serializeUser((user, done) => {
  done(null, { name: user.name, token: user.token });
});
passport.deserializeUser((userInfo, done) => {
  done(null, userInfo); // disponível em req.user
});
```

Login

```
app.post('/login', (req, res, next) => {
  const { username, password } = req.body;
  const user = USERS.find(u => u.name===username && u.password===password);
  if (user) return req.login(user, err => err ? next(err) : res.redirect('/'));
  return res.status(401).send('Invalid credentials');
});
```

Proteger Recursos

```
function authenticate(req, res, next) {
  if (!req.isAuthenticated()) return res.redirect('/login');
  next();
}
app.get('/', authenticate, home);
```

Logout

```
app.post('/logout', (req, res) => {
  req.logout(() => res.redirect('/'));
});
```

Signup

```
app.post('/signup', (req, res, next) => {
  const { username, password } = req.body;
  if (USERS.find(u => u.name===username))
    return res.status(400).send(username + ' already exists!');
  const user = { id: nextId(), name: username, password };
  USERS.push(user);
  req.login(user, err => err ? next(err) : res.redirect('/'));
});
```

Lesson — Ligações entre Lessons — Encadeamento Temático

Visão Global da Progressão

Lessons 01–05	Fundamentos JavaScript: Tipos, operadores, estruturas de controlo, funções, objetos, arrays, higher-order functions
Lessons 06–07	Prática e Modularização: Exercícios práticos de JS; módulos ESM/CJS; NPM; testes com Mocha
Lessons 08–09	Assincronia: Callbacks → Promises → async/await; fetch API; fs/promises; concorrência
Lessons 10	Web e HTTP: Fundamentos da WWW; URI/URL; protocolo HTTP; métodos e status codes
Lessons 11–12	Express e REST API: Framework Express; routing; CRUD Tasks API; REST constraints; OpenAPI
Lessons 13–15	Qualidade da API: Compliance OpenAPI; gestão de erros; middleware; CORS
Lessons 16–17	Persistência e Arquitetura: Elasticsearch como BD; Dependency Injection para modularidade
Lessons 18–19	Frontend: HTML/CSS; Bootstrap; Handlebars template engine; views e partials
Lessons 20	Segurança: Autenticação com Passport; sessões; cookies; login/logout/signup

Evolução da Tasks API (Caso de Estudo)

- **v0:** CRUD básico em memória; sem utilizadores
- **v1.0.0:** Módulos separados; compliance OpenAPI; token Bearer; status 401
- **v1.1.0:** Módulo de dados assíncrono (Promises); gestão de erros interna → HTTP
- **v1.2.0:** Backend Elasticsearch (persistência real)
- **v1.3.0:** Dependency Injection; testes unitários com Mocha
- **v2.0.0:** Tasks Web Site com HBS; formulários HTML; redirect após ação

Lesson

Glossário — Termos-Chave

Arrow Function Sintaxe compacta de função em JS: <code>x => x*x</code>	JSON JavaScript Object Notation — formato de serialização de dados
async/await Sintaxe ES2017 para escrever código assíncrono com aparência síncrona	Middleware (Express) Função com acesso a req, res, next no ciclo request-response
Bearer Token Credencial de autenticação enviada no header Authorization	Mocha Framework de testes para Node.js (describe + it + assert)
Bootstrap Framework CSS responsivo de componentes UI prontos a usar	Node.js Runtime JavaScript do lado do servidor
Callback Função passada como argumento para ser chamada quando um evento ocorre	NPM Node Package Manager — gestão de dependências e scripts
Closure Função que captura variáveis do âmbito que a envolve	OpenAPI Especificação YAML/JSON para descrever REST APIs (ex-Swagger)
CORS Mecanismo HTTP que permite requests cross-origin controladas	Passport Middleware de autenticação para Node.js
Constructor Function Função usada com new para criar objetos de um tipo	Promise Objeto que representa resultado futuro de operação assíncrona
Coerção de Tipos Conversão implícita de tipos em JS (evitar com <code>===</code>)	Prototype Objeto base de onde objetos JS herdam propriedades e métodos
CRUD Create, Read, Update, Delete — operações básicas de dados	REST Estilo arquitetural para sistemas hipermedia (6 constraints)
Dependency Injection Padrão onde dependências são fornecidas externamente ao módulo	Scope Região do código onde uma variável é visível/acessível
Elasticsearch Motor de busca e análise distribuído, baseado em JSON e REST	Session Estado mantido entre requests via cookie no browser
ESM ECMAScript Modules — sistema de módulos padrão do JS moderno	Single-threaded JS executa num único thread (sem paralelismo real)
Express Framework minimal de routing e middleware para Node.js	Status Code HTTP Código numérico na resposta HTTP (2xx sucesso, 4xx erro cliente, 5xx erro servidor)

fetch() API global para fazer requests HTTP, retorna Promise	Template Engine Combina template HTML com dados para gerar HTML final
Handlebars (HBS) Template engine com sintaxe {{variavel}} para gerar HTML	typeof Operador JS que retorna o tipo de um valor como string
Higher-Order Function Função que recebe ou retorna outra função	URI/URL/URN Identificadores de recursos web (URI=geral, URL=localização, URN=nome persistente)
HTTP Hypertext Transfer Protocol — protocolo stateless cliente-servidor	UUID Universally Unique Identifier — identificador de 128 bits (crypto.randomUUID())
Idempotente N chamadas idênticas produzem o mesmo estado no servidor que 1 chamada	WWW World Wide Web — espaço de informação com recursos identificados por URIs